

Module 21: Stored Procedures

A Stored Procedure is a precompiled collection of SQL statements stored in the database that can be executed whenever needed. Stored Procedures help improve code reusability, security, maintainability, and performance.



Introduction

Suppose a company generates the same employee report every day:

```
SELECT EmployeeID,  
        EmployeeName,  
        Salary  
FROM Employees  
WHERE DepartmentID = 1;
```

Instead of writing this query repeatedly, we can save it as a Stored Procedure.

Then execute it using:

```
EXEC ProcedureName;
```



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand Stored Procedures
 - ✓ Create Procedures
 - ✓ Execute Procedures
 - ✓ Use Input Parameters
 - ✓ Use Output Parameters
 - ✓ Use Variables
 - ✓ Implement Error Handling
 - ✓ Understand Business Applications
-



Table of Contents

1. What is a Stored Procedure?
 2. Why Use Stored Procedures?
 3. Benefits of Stored Procedures
 4. Creating Procedures
 5. Executing Procedures
 6. Input Parameters
 7. Output Parameters
 8. Variables in Procedures
 9. Conditional Logic
 10. Error Handling
 11. Altering Procedures
 12. Dropping Procedures
 13. Best Practices
 14. Interview Questions
 15. Business Analytics Examples
 16. Summary
 17. Practice Questions
-

1 What is a Stored Procedure?

A Stored Procedure is a named collection of SQL statements stored inside the database.

Similar To

Programming:

Function
Method
Reusable Block

SQL:

CREATE PROCEDURE

Example

```
CREATE PROCEDURE GetEmployees
AS
BEGIN

    SELECT *
    FROM Employees;

END;
```

Execute:

```
EXEC GetEmployees;
```

Result:

Returns all employees.

Why Procedures Matter

Used in:

- Enterprise Applications
 - Banking Systems
 - ERP Software
 - CRM Systems
 - Reporting Platforms
-

2 Why Use Stored Procedures?

Without Procedure:

```
SELECT *
FROM Employees;
```

must be written repeatedly.

With Procedure:

```
EXEC GetEmployees;
```

Benefits:

- ✓ Reusability
 - ✓ Faster Development
 - ✓ Centralized Logic
 - ✓ Easier Maintenance
-

Example

Salary report needed daily.

Store logic once.

Execute whenever required.

3 Benefits of Stored Procedures

Reusability

Write once.

Use many times.

Security

Users execute procedures.

They don't need direct table access.

Maintainability

Modify procedure once.

Changes apply everywhere.

Performance

SQL Server caches execution plans.

Consistency

Everyone uses same business logic.



Creating Procedures

Syntax

```
CREATE PROCEDURE ProcedureName
AS
BEGIN

    SQL Statements

END;
```

Example

```
CREATE PROCEDURE GetEmployees
AS
BEGIN

    SELECT *
    FROM Employees;

END;
```

Execute

```
EXEC GetEmployees;
```

Output:

All employee records.

Another Example

```
CREATE PROCEDURE GetDepartments
AS
BEGIN

    SELECT *
    FROM Departments;

END;
```

5 Executing Procedures

Use:

```
EXEC
```

or

```
EXECUTE
```

Example

```
EXEC GetEmployees;
```

Alternative

```
EXECUTE GetEmployees;
```

Both work.

6 Input Parameters

Procedures can accept values.

Syntax

```
CREATE PROCEDURE ProcedureName
(
    @Parameter DataType
)
AS
BEGIN

END;
```

Example

```
CREATE PROCEDURE GetEmployeeByID
(
    @EmployeeID INT
)
AS
BEGIN

    SELECT *
    FROM Employees
    WHERE EmployeeID = @EmployeeID;

END;
```

Execute

```
EXEC GetEmployeeByID 101;
```

Result

Employee with ID 101.

Multiple Parameters

```
CREATE PROCEDURE GetEmployee
(
    @DepartmentID INT,
    @Salary INT
)
AS
BEGIN

    SELECT *
    FROM Employees
    WHERE DepartmentID = @DepartmentID
        AND Salary > @Salary;

END;
```

Execute

```
EXEC GetEmployee 1,50000;
```

Output Parameters

Return values from procedures.

Example

```
CREATE PROCEDURE EmployeeCount
(
    @TotalEmployees INT OUTPUT
)
AS
BEGIN

    SELECT @TotalEmployees =
        COUNT(*)
    FROM Employees;

END;
```

Execution


```
DECLARE @Count INT;  
  
EXEC EmployeeCount @Count OUTPUT;  
  
SELECT @Count;
```

Output

```
150
```

Uses

- Counts
- Totals
- Status Codes
- Validation Results

8 Variables in Procedures

Variables store temporary values.

Syntax

```
DECLARE @Variable DataType;
```

Example

```
CREATE PROCEDURE AverageSalary  
AS  
BEGIN  
  
    DECLARE @AvgSalary DECIMAL(10,2);  
  
    SELECT @AvgSalary =  
        AVG(Salary)  
    FROM Employees;
```

```
SELECT @AvgSalary;  
  
END;
```

Execute

```
EXEC AverageSalary;
```

Output

Average salary.

Why Variables?

Used for:

- Calculations
 - Conditions
 - Intermediate Results
-



Conditional Logic

Stored Procedures support:

```
IF  
ELSE
```

Example

```
CREATE PROCEDURE CheckSalary  
(  
    @Salary INT  
)  
AS  
BEGIN  
  
    IF @Salary > 50000  
  
        PRINT 'High Salary';
```

```
ELSE  
  
    PRINT 'Low Salary';  
  
END;
```

Execute

```
EXEC CheckSalary 60000;
```

Output

```
High Salary
```

10 Error Handling

Professional procedures use:

```
TRY  
CATCH
```

Example

```
BEGIN TRY  
  
    SELECT 10 / 0;  
  
END TRY  
  
BEGIN CATCH  
  
    PRINT 'Error Occurred';  
  
END CATCH
```

Result

Error Occurred

instead of application failure.

Full Example

```
CREATE PROCEDURE SafeDivision
(
    @A INT,
    @B INT
)
AS
BEGIN

    BEGIN TRY

        SELECT @A / @B;

    END TRY

    BEGIN CATCH

        PRINT 'Division Error';

    END CATCH

END;
```

Altering Procedures

Modify existing procedure.

Syntax

```
ALTER PROCEDURE ProcedureName
AS
BEGIN

END;
```

Example

```
ALTER PROCEDURE GetEmployees
AS
BEGIN

    SELECT EmployeeID,
           EmployeeName
    FROM Employees;

END;
```

Procedure updated.

1 2 Dropping Procedures

Remove procedure.

Syntax

```
DROP PROCEDURE ProcedureName;
```

Example

```
DROP PROCEDURE GetEmployees;
```

Result

Procedure removed.

Tables remain unchanged.

1 3 Best Practices

Use Meaningful Names

Good:

```
GetEmployeeByID  
GenerateSalesReport  
CalculateBonus
```

Use Parameters

Avoid hardcoding values.

Include Error Handling

Use:

```
TRY  
CATCH
```

Keep Procedures Focused

One purpose per procedure.

Document Business Logic

Essential in enterprise projects.

Avoid SELECT *

Good:

```
SELECT EmployeeID,  
       EmployeeName
```

Professional Example

```
CREATE PROCEDURE GetDepartmentEmployees  
(
```

```
    @DepartmentID INT
)
AS
BEGIN

    SELECT EmployeeID,
           EmployeeName,
           Salary

    FROM Employees

    WHERE DepartmentID = @DepartmentID;

END;
```

1

4

Common Interview Questions

What is a Stored Procedure?

A precompiled collection of SQL statements stored in the database.

Why Use Stored Procedures?

Reusability, security, performance.

Difference Between Procedure and Query?

Query runs once.

Procedure can be stored and reused.

Can Procedures Accept Parameters?

Yes.

Can Procedures Return Values?

Yes, using OUTPUT parameters.

How Do You Execute a Procedure?

```
EXEC ProcedureName;
```

How Do You Modify a Procedure?

```
ALTER PROCEDURE
```

1 **5** Business Analytics Examples

Employee Lookup

```
EXEC GetEmployeeByID 101;
```

Department Report

```
EXEC GetDepartmentEmployees 1;
```

Sales Dashboard Procedure

```
CREATE PROCEDURE SalesSummary
AS
BEGIN

    SELECT ProductID,
           SUM(SalesAmount)

    FROM Sales

    GROUP BY ProductID;

END;
```


KPI Procedure

```
CREATE PROCEDURE EmployeeKPI
AS
BEGIN

    SELECT COUNT(*) AS Employees,
           AVG(Salary) AS AvgSalary

    FROM Employees;

END;
```

Monthly Reporting

Common in enterprise environments.



Summary

In this module, you learned:

- ✓ Stored Procedures
 - ✓ CREATE PROCEDURE
 - ✓ EXECUTE PROCEDURE
 - ✓ Input Parameters
 - ✓ Output Parameters
 - ✓ Variables
 - ✓ IF ELSE Logic
 - ✓ Error Handling
 - ✓ ALTER PROCEDURE
 - ✓ DROP PROCEDURE
-



Practice Questions

Theory

1. What is a Stored Procedure?
 2. Why use Stored Procedures?
 3. How do you create a Procedure?
 4. How do you execute a Procedure?
 5. What are Input Parameters?
 6. What are Output Parameters?
 7. How do Variables work?
 8. Why use TRY-CATCH?
 9. How do you modify a Procedure?
 10. How do you delete a Procedure?
-

Practical Exercises

Task 1

Create:

```
GetEmployees
```

Procedure.

Task 2

Create a parameterized procedure.

Task 3

Create a procedure returning employee count.

Task 4

Use variables inside a procedure.

Task 5

Implement TRY-CATCH.

Task 6

Modify a procedure using:

```
ALTER PROCEDURE
```

Challenge Project

Create:

```
Employees
Departments
Sales
Orders
Customers
```

Build procedures for:

- Employee Reports
- Sales Summary
- Customer Analytics
- KPI Dashboard
- Error-Handled Reporting



Next Module

→ Module 22: SQL Functions

Topics Covered:

- Built-in Functions
- Scalar Functions
- Aggregate Functions
- String Functions
- Date Functions
- Mathematical Functions
- User-Defined Functions (UDFs)
- Function vs Procedure
- Analytics Use Cases